Date

Expt. No. 5.

Page No. 22.

Aim:- To compute shortest path ~~between~~ between all pairs of vertices using Floyd's algorithm.

Theory:- Floyd's algorithm is a Dynamic Programming algorithm used to find the shortest paths between all vertices in a weighted graph. It updates the distance matrix by considering each vertex as an intermediate vertex. It works with negative edge weights but not with negative-weight cycles.

Algorithm:-

1. FLOYD (A, n)
2. D = A
3. FOR k = 1 TO n
   FOR i = 1 TO n
   FOR j = 1 TO n
   IF D[i][k] + D[k][j] < D[i][j]
   D[i][j] = D[i][k] + D[k][j]
4. PRINT D
5. STOP

Complexity:-

Time Complexity = $O(N^3)$

Space Complexity = $O(n^2)$

Teacher's Signature

Date

Expt. No. Page No. 23.

C++ Program :-

```
#include <bits/stdc++.h>
using namespace std;
const int INF = 1e9;
int main() {
    int n = 4;
    vector<vector<int>> dist = {
        { 0, 5, INF, 10 },
        { INF, 0, 3, INF },
        { INF, INF, 0, 1 },
        { INF, INF, INF, 0 }
    };

for(int k=0; k<n; k++)
    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++)
            if( dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];
for(auto &row : dist)
    {
for(int x : row) cout << (x == INF? -1 : x)
    << " "; cout << "\n"; }
return 0;
}
```

Teacher's Signature

Date

Expt. No. Page No. 24.

Viva Question

Q1 What is Floyd's Algorithm used for?

Ans It is used to find the shortests paths between all pairs of vertices in a weighted graphs.

Q2 Which technique is used in Floyd's Algorithm?

Ans It uses the Dynamic Programming technique.

Q3 What is the time Complexity of Floyd's Algorithm.

Ans It's time complexity is $O(n^3)$.

Q4. Can Floyd's Algorithm handle negative edge weights.

Ans Yes, it can handle negative edge weights, but not negative-weight cycles.

Q5 What is the main difference between Dijkstra's & Floyd's Algorithm?

Ans Dijkstra's Algorithm finds shortest path from a single source, whereas Floyd's algorithm finds shortest path between all pairs of vertex.

Teacher's Signature

Date

Expt. No. 6. Page No. 25.

Aim:- To compute single-source shortest paths in a weighted ~~pal~~ graphs.

Theory:- Dijkstra's ~~use~~ Algorithm is a greed algorithm used to find the shortest paths from a single source vertex to all other vertex in a weighted graph. It repeatedly selects the vertex with the minimum distance & updates the distances of its adjacent vertex. It works only with non-negative edge weights.

Algorithm:-

DIKJSTRA(G, source)

1. FOR each vertex v:
   dist[v] = INFINITY
   visited[v] = FALSE;
2. dist[source] = 0
3. WHILE there are unvisited vertices:
   u = unvisited vertex with minimum dist[u]

   IF dist[u] == INFINITY:
   BREAK
   visited[u] = TRUE
   FOR each adjacent vertex v of u:
   IF visited[v] == FALSE AND
   dist[u] + weight(u,v) < dist[v];
   dist[v] = dist[u] + weight(u,v)
4. PRINT dist[]
5. Stop.

Teacher's Signature

Date

Expt. No. Page No. 26.

C++ Program :-

```
#include <bits/stdc++.h>
using namespace std;
using pii = pair<int,int>;
const int INF = 1e9;

void dijkstra(int V, vector<vector<pii>>
                &adj, int src) {
  vector<int> dist(V, INF);
  dist[src] = 0;
  priority_queue<pii, vector<pii>, greater<pii>>
                pq;
  pq.push({0, src});
  while (!pq.empty()) {
    auto [d,u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;
    for (auto [v,w] : adj[u]) {
      if (dist[v] > dist[u] + w) {
        dist[v] = dist[u] + w;
        pq.push({dist[v], v});
      }
    }
  }
  for(int i=0; i<V; i++) cout << i << ":"
        << dist[i] << "\n";
}
int main() {
  int V=5;
```

Teacher's Signature

```
vector<vector<pii>> adj(V);
adj[0] = {{1,9},{2,6},{3,5},{4,3}};
adj[0].push_back({1,2});
adj[2].push_back({3,4});
dijkstra(V, adj, 0);
return 0;
}
```

Complexity :-

Time Complexity = $O(V^2)$

Space Complexity = $O(V)$

Viva Question

Q1 What is Dijkstra's Algorithm used for?

Ans It finds the shortest paths from a single source vertex to all other vertices in a weighted graph.

Q2 Which technique is used in Dijkstra's algorithm?

Ans It uses the greedy approach.

Q3 Can Dijkstra's algorithm handle negative edge weights?

Ans NO, Dijkstra's algorithm does not work ~~directly~~ correctly with negative edge weights in general.

Teacher's Signature _____

Q4 What is the time complexity of Dijkstra's Algorithm?

Ans The is $O(V^2)$ using an array-based implementation, where V is the number of vertices.

Q5 What is relaxation in Dijkstra's algorithm?

Ans Relaxation means updating a vertex's distance if a shorter path is found through another vertex.

Date

Expt. No. 7. Page No. 29.

Aim :- To Compute MST of an undirected graph using Kruskal's algorithm.

Theorem :- Kruskal's algorithm is a greedy Algorithm used to find the minimum spanning tree (MST) of a connected, undirected, weighted graph. It sorts all edges in increasing order of weighted & selects the smallest edge that does not form a cycle, until V−1 edges are selected.

Algorithm :-

KRUSKAL (G, V, E)

1. Sort all edges in increasing order of weights
2. MST = empty set
   TotalCost = 0
3. Initialize each vertexs as a seperate set.
4. FOR each edge (u, v) in sorted edges:
   IF FIND (u) != FIND (v);
   Add (u, v) to MST
   TotalCost += weight (u, v)
   UNION (u, v)
   IF MST Contains V−1 edges:
   BREAK
5. PRINT MST and TotalCost.
6. STOP.

Teacher's Signature

Date

Expt. No. Page No. 86

Source Code :-

```
#include <bits/stdc++.h>
using namespace std;
struct DSU { vector<int> p, r;
DSU(int n) : p(n), r(n) { iota(p.begin(),
    p.end(), 0); }
int find(int x) {
    return p[x] == x ? x : p[x] = find(p[x]);
    }

bool unite(int a, int b) {
    a = find(a);
    b = find(b);
  if(a == b) {
    return false;
    }
 if (r[a] < r[b])
    swap(a,b);

 p[b] = a;
 if (r[a] == r[b])
    r[a]++;
 return true;
}
};
```

Complexity

Time Complexity = O(E log E)

Space Complexity = O(V) for the DSU, excluding graph storage.

Teacher's Signature

```
int main() {
    int V = 4;
    vector<tuple<int,int,int>> edges = {
        {0,1,10},
        {0,2,6},
        {0,3,5},
        {1,3,15},
        {2,3,4}
    };

sort(edges.begin(), edges.end(),
    [](auto &a, auto &b) {
        return get<2>(a) < get<2>(b);
    });
DSU dsu(V);
int cost = 0;
int count = 0;

cout << "Edges in Minimum Spinning Tree:\n";
for(auto &e : edges)
    int u,v,w;
    tie(u,v,w) = e;
    if(dsu.unite(u,v)) {
        cost += w;
        count++;
    cout << u << "-" << v
        << " (" << w << ")\n";
    if(count == V-1)
        break;
    }
}
```

Teacher's Signature

```
if (count != V-1) {
    cout << "MST does not exist. Graph is
    disconnected :\n";
} else {
    cout << "Total MST Cost :" << cost
         << "\n";
}
return 0;
}
```

Viva Question

Q1 What is Kruskal's Algorithm?

Ans Kruskal's Algorithm is a greedy algorithm used to find the Minimum spining tree (MST) of a weighted, undirected graph.

Q2 What is a Minimum Spanning Tree (MST)?

Ans An MST connects all vertices with the minimum total edge weight without forming any cycle.

Q3 Why do we sort edges in Kruskal's algorithm?

Ans We sort edges in increasing order of weight so that we can selects the minimum - cost edges first.

Q4 What is the role of DSU (Disjoint Set Union)?

Ans DSU helps detect cycles by checking whether two vertices belong to the same set.

Teacher's Signature

Date

Expt. No.

Page No. 33

Q5 What is the time complexity of Kruskal's algorithm?

Its time complexity is $O(E \log E)$, where E is the number of edges.

Teacher's Signature